

Rajiv Gandhi University of Knowledge and Technologies
Srikakulam



Digital Clock, Timer and Stopwatch
Implementation Using FPGA

PRESENTED BY

V.V.V.SAI SRI(S200093)

R. HARIKA (S200378)

S. JHANSI (S200529)

S.N.S.N. REDDY(S200542)

S. KAVYA SRI(S201019)

P. SAIVAMSI(S201068)

UNDER THE GUIDANCE OF

Mrs. K. SUDHARANI, MTech

Department of ECE

Table of content

1.Abstract.....	3
2.Introduction.....	4-5
3.FPGA Board and its Components.....	6-8
• FPGA Chip	
• Clock source	
• Power supply	
• I/O Ports	
• Memory	
• Programming Interface	
• Displays	
• Communication Ports	
• Expansion Headers	
3.7 Segment Display.....	9-12
• What is a 7-Segment Display?	
• Applications of 7-Segment Display	
• Advantages	
• Limitations	
• Interfacing with Microcontrollers	
4.Digital Clock.....	13-16
5.Timer.....	17-19
• Hardware based	
• Software based	
• Applications	
• Advantages	
• Extra features	
6.Stopwatch.....	20-22
• Hardware based	
• Software based	
• Applications	
• Advantages	
• Extra features	
7.Applications.....	23-24
8.Conclusion.....	42

ABSTRACT

The Basys 3 FPGA development board, based on the Xilinx Artix-7 FPGA, offers a versatile platform for implementing various digital systems and applications. Among these, time-based systems such as digital clocks, timers, and stopwatches are classic and fundamental examples for understanding digital design concepts like counters, state machines, and seven-segment display control.

In this project, we implement a digital clock, timer, and stopwatch using the Basys 3 board. These systems rely on precise time counting and display mechanisms, making them ideal for learning about clock division, multiplexing, and hardware description languages such as VHDL or Verilog.

The digital clock continuously tracks and displays the current time in hours, minutes, and seconds format.

The timer allows a user to set a countdown value and triggers an alert or signal when the countdown reaches zero.

The stopwatch measures elapsed time from a starting point and can be paused, reset, and resumed, making it useful for timing specific events.

These applications demonstrate key FPGA design principles, including the use of clock dividers, counters, state machines, and interfacing with input/output peripherals such as pushbuttons, switches, and the onboard seven-segment displays of the Basys 3 board.

Introduction:

In the realm of digital system design, creating functional timekeeping devices such as clocks, stopwatches, and timers represents a fundamental application of sequential logic and modular hardware design. These systems are excellent exercises for understanding the principles of synchronous design, including clock signal management, counters, finite state machines (FSMs), and digital displays.

The Basys-3 FPGA development board, built around the Xilinx Artix-7 FPGA, is a widely used educational platform that supports a variety of digital logic design experiments. It includes essential components such as a 100 MHz clock source, push buttons, switches, LEDs, and four 7-segment displays, all of which are ideal for building user-interactive time-based systems.

Implementing timekeeping functions on an FPGA, rather than a microcontroller, gives students and engineers deeper insight into low-level hardware behavior. Unlike software-based solutions, FPGA designs involve direct manipulation of digital logic through hardware description languages (HDLs) like Verilog or VHDL, leading to faster and deterministic performance.

The Basys-3 board, based on the Xilinx Artix-7 FPGA, offers an ideal platform for learning and prototyping digital logic designs. Implementing a clock, stopwatch, and timer allows students and engineers to explore the use of finite state machines (FSMs), counters, multiplexing, and synchronous digital design.

Relevance and Applications

Designing these systems has both educational and real-world value:

- Clocks are fundamental in all digital systems, including real-time clocks in embedded systems.
- Stopwatches are used in sports, laboratories, and industrial timing systems.
- Timers are essential in cooking devices, countdown applications, and time-based automation.

By designing them on an FPGA:

- You develop skills in modular design.
- You gain hands-on experience with timing constraints, hardware-level I/O interfacing, and sequential control logic.
- You understand how hardware processes time-related events independent of software layers.
- Technical Objectives

The design typically includes:

Clock Divider: To generate lower-frequency signals from the 100 MHz master clock.

Counters: To count time intervals for seconds, minutes, etc.

Control Logic: Using FSMs to manage start, stop, reset, and set conditions.

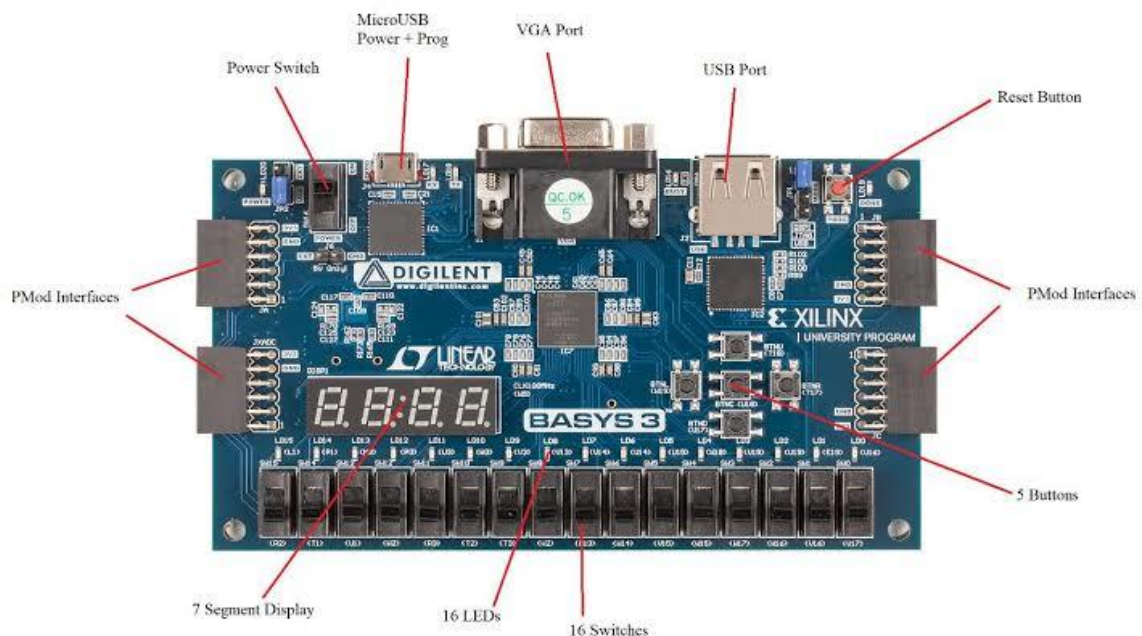
Debouncing: Handling mechanical noise from buttons and switches.

Display Management: Driving the Basys-3's 7-segment displays using multiplexing techniques.

FPGA Basis Board and Its Components:

An FPGA (Field Programmable Gate Array) is an integrated circuit that can be programmed by the user after manufacturing. It is widely used in digital electronics because of its flexibility and reusability. An FPGA Basis Board is a development board that contains an FPGA chip and various other components to help students, engineers, and developers test and build digital circuits or systems.

The main purpose of an FPGA basis board is to make it easier to design, test, and implement digital logic circuits. It allows users to create their own digital systems using hardware description languages like VHDL or Verilog. Once the design is ready, it can be uploaded to the FPGA chip on the board.



Let's look at the key components found on a typical FPGA basis board:

FPGA Chip:

This is the heart of the board. It contains thousands of logic blocks that can be connected in different ways to perform specific digital tasks. Users can program the chip to behave like any digital circuit, such as a calculator, processor, or controller.

Power Supply:

Every board needs a power supply to function. Most boards have voltage regulators to convert the input power into levels suitable for the FPGA chip and other components.

Clock Source:

A clock is needed to synchronize all operations inside the FPGA. The board usually has a crystal oscillator that provides a steady clock signal for the chip.

Input/Output (I/O) Ports:

These ports allow the FPGA to communicate with the outside world. Common I/O components include switches, push buttons, and LEDs. Users can interact with their design using these inputs and see outputs on LEDs or display screens.

Memory:

Some boards include external memory like RAM or ROM. These are used when your design needs to store data temporarily or permanently.

Programming Interface:

This is used to load the design into the FPGA. A USB or JTAG connector is usually provided to connect the board to a computer for programming.

Displays:

Many boards come with seven-segment displays or small LCDs to show output values. This is helpful for testing and debugging designs.

Communication Ports:

Some boards have communication interfaces like UART, SPI, or I2C. These allow the FPGA to talk with other devices like sensors, microcontrollers, or computers.

Expansion Headers:

These are pin connectors that allow users to connect external devices or custom circuits to the board for more complex projects.

Using an FPGA basis board is a great way to learn how hardware works. Unlike software programming, where code runs on a processor, FPGA programming creates actual hardware behavior. This gives students and developers real experience with digital electronics and system design.

In summary, the FPGA basis board is a powerful tool used in electronics and computer engineering. It provides a platform to design, test, and implement digital systems in real hardware, making it essential for learning and innovation in the field of digital design.

7-Segment Display

A 7-segment display is one of the most common and easy-to-use electronic display devices. It is widely used in digital clocks, calculators, measuring instruments, and other devices that show numerical information. It gets its name because it is made up of seven individual segments (LEDs), which can be turned on or off to display numbers and some basic characters.



What is a 7-Segment Display?

A 7-segment display is an electronic component made of seven light-emitting diodes (LEDs) arranged in a specific pattern. These segments are labeled as a, b, c, d, e, f, and g, and are usually arranged in a rectangular fashion to form numbers from 0 to 9. Some displays also include a dot (.), known as the decimal point (dp).

Each segment glows when electricity passes through it. By turning on specific segments, you can display any digit from 0 to 9. For example:

To display the digit 2, segments a, b, g, e, and d are turned on.

To display 8, all seven segments are turned on.

Types of 7-Segment Displays

There are mainly two types of 7-segment displays based on how the LEDs are connected inside:

Common Cathode (CC):

In a common cathode display, all the negative terminals (cathodes) of the LEDs are connected together to ground.

To light up a segment, you apply a HIGH voltage (logic 1) to its pin.

Common Anode (CA):

In a common anode display, all the positive terminals (anodes) are connected together to the power supply.

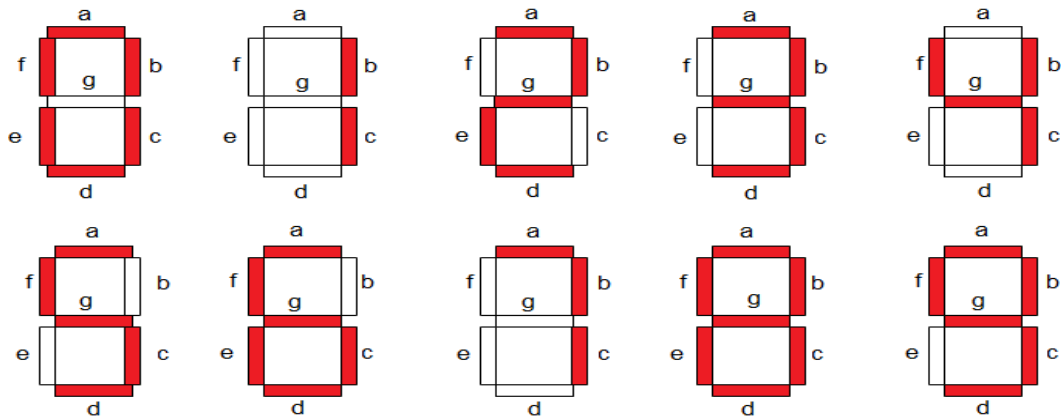
To light up a segment, you connect its pin to LOW voltage (logic 0 or ground).

How It Works

Each segment in the display is controlled individually using electrical signals. A microcontroller, FPGA, or any digital circuit sends signals to the display to control which segments light up. For example, to show the number 3, you turn on segments a, b, c, d, and g.

Here's a simple mapping of segments needed to display each digit:

Digit	Segments On
0	a, b, c, d, e, f
1	b, c
2	a, b, g, e, d
3	a, b, c, d, g
4	f, g, b, c
5	a, f, g, c, d
6	a, f, e, d, c, g
7	a, b, c
8	a, b, c, d, e, f, g
9	a, b, c, d, f, g



Applications of 7-Segment Display

7-segment displays are used in many devices because they are simple and easy to read. Some common applications include:

- Digital clocks
- Calculators
- Microwave ovens
- Gas pumps
- Electronic meters
- Scoreboards

Advantages:

- Simple to use
- Low cost
- Easy to read
- Requires less wiring for basic projects

Limitations:

- Can only display numbers and a few letters (like A, b, C, d, E, F)
- Not suitable for displaying complex text
- Limited brightness and color (unless advanced models are used)

Interfacing with Microcontrollers

To control a 7-segment display, you can connect it to a microcontroller like Arduino, Raspberry Pi, or an FPGA board. You can either connect it directly or use driver ICs like 7447 or 4511, which simplify the control of the segments.

You can also use multiplexing when controlling multiple 7-segment displays. This technique turns each display on and off rapidly so that they appear to be on all at the same time. This saves pins and power.

Conclusion

The 7-segment display is a basic but very useful electronic component for showing numbers in digital form. It is easy to use, inexpensive, and perfect for beginners learning about digital electronics. Understanding how it works is an important step toward working with more advanced display systems in the future.

Digital Clock :

A digital clock is a device that shows the time using digits, rather than hands on a dial like an analog clock. It usually displays hours, minutes, and seconds in a clear and easy-to-read format, often as HH:MM:SS. In today's world, digital clocks are very common and can be found in homes, offices, cars, and electronic devices like computers and mobile phones. This mini project is focused on designing and implementing a digital clock using basic electronics or programming tools.



The main goal of this project is to create a working digital clock that can keep and display accurate time. Depending on the method used, it can be built using electronic components such as a microcontroller (like Arduino), a display module (like a 7-segment display or LCD), and a real-time clock (RTC) module to maintain time even when the power is off.

Alternatively, the project can also be done using programming languages like Python, C++, or Java to create a software-based digital clock that runs on a computer.

In a hardware-based digital clock, the RTC module plays a very important role. It keeps track of the time independently and sends the time information to the microcontroller. The microcontroller then processes this data and shows it on the display screen. The display is

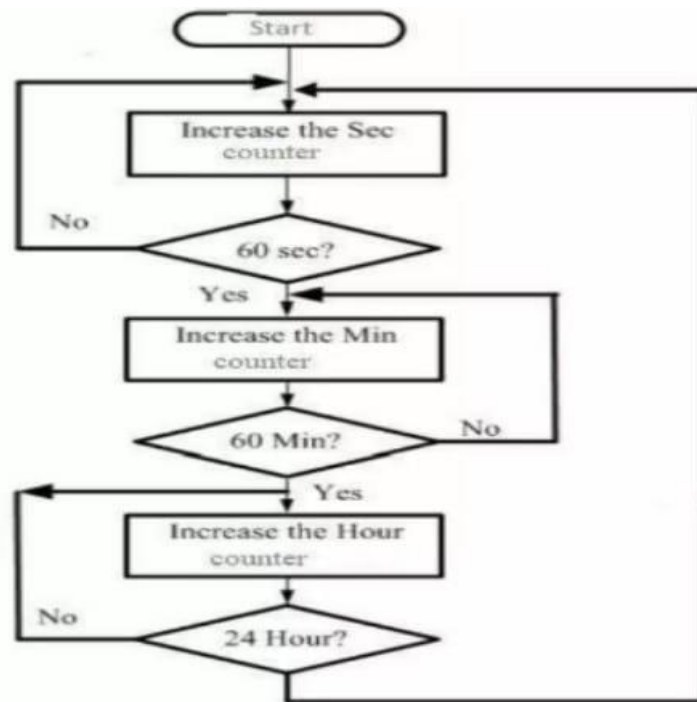
updated every second to keep the time accurate. A power supply or batteries are used to run the system.

In a software-based digital clock, the system uses the computer's internal clock to get the current time. The program reads the time continuously and updates the display on the screen in real time. This type of clock is useful for learning how to work with time functions, loops, and graphical displays in programming.

The project also helps students understand various concepts such as timekeeping, electronics, coding logic, user interface design, and the importance of accurate timing in digital systems. Optional features can also be added to make the project more interesting, such as alarm settings, date display, or stopwatch functions.

One of the main benefits of this project is that it gives hands-on experience with both hardware and software (depending on the method used). It also improves problem-solving and troubleshooting skills, as students may need to debug errors in wiring or code. Digital clocks may look simple, but they are a great example of how different technologies work together to create a useful tool.

In conclusion, building a digital clock is a fun and educational mini project. It is simple enough for beginners but also offers many ways to expand and customize. Whether done with physical components or as a computer program, the project teaches valuable skills and gives a sense of accomplishment when the final clock is running and showing the correct time.



This flowchart represents the logic of a digital clock—specifically, how seconds, minutes, and hours are incremented in a 24-hour clock system. Let's go step-by-step through each part of the diagram to explain its operation in detail.

Flowchart Explanation

1. Start

The process begins at the "Start" block.

This is typically where system initialization would happen (e.g., resetting all counters to 0).

2. Increase the Seconds Counter

Every second (from the system clock or a 1 Hz pulse), the seconds counter is incremented by 1.

This is usually handled by a clock divider that generates a 1 Hz signal from the 100 MHz Basys-3 board clock.

3. Check: 60 Seconds?

If the seconds counter reaches 60, it resets to 0, and the minutes counter is incremented.

If it hasn't reached 60, the system loops back to continue counting seconds.

4. Increase the Minutes Counter

This step is executed only when seconds = 60.

The minutes counter is incremented by 1.

5. Check: 60 Minutes?

When the minutes counter reaches 60, it resets to 0, and the hours counter is incremented.

If it hasn't reached 60, it loops back to continue normal operation.

6. Increase the Hours Counter

This step occurs only when minutes = 60.

The hours counter is incremented by 1.

7. Check: 24 Hours?

If the hours counter reaches 24, it resets to 0 (since this is a 24-hour clock format).

If not, the system continues counting normally.

Loop Back to Seconds

After processing the hour check, the system loops back to the seconds counter to continue real-time operation.

Key Concepts Represented

Hierarchical Counting: Seconds → Minutes → Hours

Threshold Checking: Logic gates or comparators check when 60 seconds, 60 minutes, or 24 hours are reached.

Reset and Carry Logic: On reaching thresholds, the current counter resets, and the next higher unit increments.

Continuous Loop: This ensures timekeeping continues indefinitely unless reset manually.

Timer :

A timer is a device or a tool that counts down or up for a specific period of time. It helps in measuring time intervals and is used in many day-to-day activities like cooking, exercising, studying, games, or industrial processes. In this mini project, we are going to design and implement a basic timer system that can either count down from a set time or count up from zero, depending on the purpose.



The main idea behind this project is to understand how timers work and how to build one using simple components or programming. A timer can be created in two main ways: using electronic hardware components (like Arduino, microcontrollers, or timer ICs), or by creating a software-based timer using programming languages such as Python, Java, or C++.

Hardware-Based Timer

In a hardware-based timer project, we can use components like a microcontroller (e.g., Arduino), a display module (7-segment or LCD), and buttons for setting the time or starting/stopping the timer. The timer can be programmed to start counting once the user sets a specific time. It can show the remaining time on the display and alert the user when the countdown reaches zero using a buzzer or LED.

The timer works with the help of a microcontroller that performs time calculations using its internal clock. The user interface is created using buttons or switches to allow the user to set minutes and seconds. Once started, the countdown begins, and the display updates every second. When the timer ends, the microcontroller sends a signal to trigger an alert.

This version of the project helps students learn how to use timers in microcontrollers, work with real-time input/output, and understand how digital electronics manage timing operations.

Software-Based Timer

In a software-based timer, the timer is built using a programming language on a computer or mobile device. The program displays a digital timer on the screen and allows the user to start, stop, reset, or set a specific countdown time. It can be made with a simple user interface using a graphical toolkit (like Tkinter in Python or Swing in Java) or with just console output.

This version uses the computer's internal clock or time library functions to calculate time. The timer checks the current time, subtracts it from the target time, and updates the display every second. This project helps improve programming skills and teaches how to manage time-based functions, loops, and user inputs.

Applications of Timers

- Timers are used in many areas of real life, such as:
- Kitchen timers for cooking
- Alarms and reminders
- Exercise and fitness timers
- Exam and study timers
- Countdown clocks in games or sports
- Industrial machinery control systems

Advantages of the Timer

Doing a timer project gives you a strong understanding of how time can be managed in both hardware and software systems. It also teaches you about accuracy, user interaction, and real-time system behavior. Students can also improve their debugging, problem-solving, and coding skills.

Extra Features

- To make the project more interesting, you can add extra features like:
- Sound or visual alerts
- Pause and resume buttons
- Set timer presets (e.g., 5 min, 10 min)
- Countdown and stopwatch modes
- Saving timer history

Conclusion

In conclusion, a timer is a simple but very useful device that helps in time management. Creating a timer as a mini project is a great way to learn the basics of electronics or programming, depending on the method you choose. It also gives hands-on experience with real-world applications and helps improve technical skills. Whether it is hardware-based or software-based, a timer project is easy to build, fun to use, and very educational.

A countdown timer starts from a user-set time and counts down to zero. It may include alarms or blinking displays when the countdown finishes.

Each module shares foundational concepts such as:

- Clock division (from 100 MHz to lower frequencies)
- State machines for control logic
- Display multiplexing for efficient use of 7-segment displays
- Button debouncing for stable input detection

Stopwatch :

A stopwatch is a timekeeping device that is used to measure the exact time taken for an activity or event. It is commonly used in sports, games, laboratories, cooking, and many other situations where accurate timing is important. A stopwatch can start, stop, and reset the time. Some stopwatches also have lap-time and split-time features. In this mini project, we are going to design and create a basic stopwatch using either hardware components or software programming.



The main goal of this project is to build a stopwatch that is simple, functional, and easy to use. It should be able to count time from zero and display it in the format of hours, minutes, seconds, and optionally milliseconds. The stopwatch should have basic buttons to Start, Stop, and Reset the time. This project is ideal for beginners and students who want to understand time-based systems and improve their skills in electronics or programming.

Hardware-Based Stopwatch

In a hardware-based version of this project, we can use an electronic microcontroller like Arduino, along with a display (such as an LCD screen or 7-segment display), and some push buttons for the controls. The microcontroller is programmed to track time using its internal clock and to update the display in real-time.

When the Start button is pressed, the microcontroller starts counting time in the background. When Stop is pressed, the stopwatch freezes the time. The Reset button brings the timer back to zero. The display continuously shows the updated time values so the user can easily track the time passed.

This project helps students learn how to program a microcontroller, work with digital displays, and use timers or delay functions in hardware. It also introduces them to the basics of electronic circuits and embedded systems.

Software-Based Stopwatch

In a software-based stopwatch, we can create a simple application using a programming language like Python, Java, or C++. It can be either a command-line program or a graphical user interface (GUI) application using libraries like Tkinter (in Python), Swing (in Java), or others.

The program uses the system clock to keep track of the time. When the user clicks the Start button, the program records the current time. As time passes, it calculates the difference between the current time and the start time and displays it on the screen. The user can also stop the stopwatch and reset it back to zero.

This type of project is great for learning how to work with time functions, loops, and GUI development in programming. It also helps understand how to handle real-time data updates and user inputs.

Applications of a Stopwatch

- Stopwatches are used in many different areas such as:
- Sports and athletics to time races
- Laboratories to time experiments
- Workouts and physical training sessions
- Cooking and baking
- Productivity tracking (e.g., Pomodoro technique)
- Games and quizzes

Benefits of This Project

- Working on a stopwatch project helps in understanding:
- How time is tracked and displayed
- How to handle start, stop, and reset logic
- How to interact with users through buttons or GUI
- Basic principles of timing in computers or microcontrollers
- It also builds important skills in both hardware (wiring, circuit design) and software (logic building, coding, debugging).

Extra Features

- To make your stopwatch more interesting, you can add:
- Lap time or split time functionality
- Milliseconds or centiseconds display
- Sound alerts when starting or stopping
- Save timing data to a file
- Stylish graphical interface or touchscreen display

Conclusion

In conclusion, a stopwatch is a useful and practical mini project that helps students understand how time tracking works in real-life applications. It is simple to build but also offers room for creativity and extra features. Whether you choose to build it using electronics or programming, the stopwatch project will improve your technical knowledge and problem-solving skills in a fun and hands-on way.

Applications of Digital Clock, Timer, and Stopwatch Using FPGA Board:

In this mini project, we are implementing three time-based systems – digital clock, timer, and stopwatch – using an FPGA (Field Programmable Gate Array) board. These three systems are very useful in our daily life, and building them using an FPGA helps us learn both their practical applications and the power of digital design.

FPGA boards are special devices that can be programmed to perform different digital functions. They are used in many real-world systems such as computers, communication devices, robotics, and automation. By using an FPGA board to create a digital clock, timer, and stopwatch, we get to explore how real-time systems work at the hardware level, which is faster and more efficient than software-only solutions.

Let's look at the applications of each part of this project:

1. Digital Clock Applications

A digital clock shows the current time in hours, minutes, and seconds using digital numbers. It can be found everywhere – in homes, offices, schools, vehicles, and electronic devices. The digital clock we build using FPGA can be used in:

- Wall or desk clocks for homes and offices
- Alarm clocks (with added features)
- Time displays in public places like railway stations or hospitals
- Smart devices like microwaves, ovens, and washing machines
- Embedded systems in cars, mobile phones, and computers
- Using FPGA makes the digital clock more reliable and customizable. We can also add extra features like a 12/24-hour format, alarm, or date display.

2. Timer Applications

A timer counts down from a set time and alerts the user when time runs out. It is commonly used where timing control is needed. The timer built on an FPGA can be used in:

- Cooking appliances like ovens and rice cookers
- Industrial machines to control process timings
- Automated testing of electronics and other equipment
- Traffic light systems for managing signal timing
- Classroom environments for timed tests or activities

- Fitness and exercise routines to time workouts

With an FPGA, we can design very accurate timers with flexible features. It can be reprogrammed to change the countdown time or to perform specific actions when the time is up (like turning on/off a device).

3. Stopwatch Applications

A stopwatch is used to measure how much time has passed from a starting point to an ending point. It is useful for tracking time during activities and competitions. The FPGA-based stopwatch can be used in:

- Sports events for timing races and performances
- Laboratories for experiment timing
- Workouts and training sessions
- Game shows or timed games
- Productivity tracking (like Pomodoro method)
- Educational projects where precise time measurement is needed
- An FPGA-based stopwatch works very accurately and quickly, which is perfect for high-speed or real-time applications.

Why Use FPGA?

Using an FPGA for this project has many advantages:

- **High Speed:** FPGA can process time operations faster than software.
- **Accuracy:** It uses hardware logic to manage time very precisely.
- **Flexibility:** We can modify the circuit easily without building a new one.
- **Learning Value:** It helps students learn digital design, timing circuits, counters, and finite state machines.

Building a digital clock, timer, and stopwatch using an FPGA board shows how basic time-based tools can be applied in many real-life situations. These systems are found everywhere around us, from homes to industries. This project not only teaches the working and applications of these tools but also helps us gain valuable experience in digital circuit design, FPGA programming, and real-time system implementation.

Code:

```
module digital_clock_12hours(
    seg_out,
    am_pm,
    led_out,
    load_input1,
    load_input2,
    load_input3,
    CLK100MHZ,
    reset,
    an
);
    input CLK100MHZ, reset;
    input [5:0] load_input1, load_input2;
    input [3:0] load_input3;
    output [6:0] seg_out;
    output reg [3:0] an;
    output reg [11:0] led_out;
    output am_pm;
    wire clk_out;
    wire [1:0] count;
    wire [5:0] sec_1, min_1;
    wire [3:0] hours_1;
    reg [3:0] tens, units, tens_1, units_1;
    wire display_clk;

    // Frequency divider to create a 1 Hz clock from 100 MHz
    freq fq(.clk_in(CLK100MHZ), .clk_out(clk_out));
    display_clock_divider display_clk_div(.clk_in(CLK100MHZ), .clk_out(display_clk));

    // Counter module
    count cn(
        .clk(clk_out),
        .rst(reset),
        .sec(sec_1),
        .min(min_1),
        .hours(hours_1),
        .load_input1(load_input1),
        .load_input2(load_input2),
        .load_input3(load_input3),
```

```

        .am_pm(am_pm)
    );
    // 4-bit multiplexing counter
    count_4 cnn(.clk(display_clk), .rst(reset), .count(count));

    // Update BCD values for display
    always @(*) begin
        tens = sec_1 / 10;
        units = sec_1 % 10;
        tens_1 = min_1 / 10;
        units_1 = min_1 % 10;
    end

    // LED output logic for hours
    always @(*) begin
        led_out = 12'b0; // Reset all LEDs
        if (hours_1 > 0 && hours_1 <= 12) begin
            for (integer i = 0; i < hours_1; i = i + 1) begin
                led_out[i] = 1;
            end
        end
    end

end

// Multiplexing logic for seven-segment display
reg [3:0] bcd;
always @(posedge display_clk) begin
    if (reset) begin
        an = 4'b1111;
        bcd = 4'b0000;
    end else begin
        case (count)
            2'b00: begin
                an = 4'b0111;
                bcd = tens_1; // Tens of minutes
            end
            2'b01: begin
                an = 4'b1011;
                bcd = units_1; // Units of minutes
            end
            2'b10: begin
                an = 4'b1101;

```

```

        bcd = tens; // Tens of seconds
    end
2'b11: begin
    an = 4'b1110;
    bcd = units; // Units of seconds
    end
    default: begin
        an = 4'b1111;
        bcd = 4'b0000;
    end
endcase
end
end

// Seven-segment display decoder
seven_segment ssg(.seg(seg_out), .in(bcd));
endmodule

// Frequency Divider
module freq(clk_in, clk_out);
    input clk_in;
    output reg clk_out;
    reg [26:0] counter;
    initial begin
        clk_out = 0;
        counter = 0;
    end

    always @(posedge clk_in) begin
        if (counter == 50000000) begin
            counter <= 0;
            clk_out <= ~clk_out;
        end else begin
            counter <= counter + 1;
        end
    end
end
endmodule

// Display Clock Divider
module display_clock_divider(input wire clk_in, output reg clk_out);
    reg [16:0] counter;

```

```

always @(posedge clk_in) begin
    if (counter == 10000) begin
        clk_out <= ~clk_out;
        counter <= 0;
    end else begin
        counter <= counter + 1;
    end
end
endmodule

```

```

// 4-bit counter for multiplexing
module count_4(input clk, rst, output reg [1:0] count);
    always @(posedge clk) begin
        if (rst) begin
            count <= 2'b00;
        end else if (count == 2'b11) begin
            count <= 2'b00;
        end else begin
            count <= count + 1;
        end
    end
end
endmodule

```

```

// Counter Module
module count(
    input clk,
    rst,
    input [5:0] load_input1,
    load_input2,
    input [3:0] load_input3,
    output reg [5:0] sec,
    min,
    output reg [3:0] hours,
    output reg am_pm
);
    always @(posedge clk) begin
        if (rst) begin
            sec <= (load_input1 < 60) ? load_input1 : 0;
            min <= (load_input2 < 60) ? load_input2 : 0;
            hours <= (load_input3 > 0 && load_input3 <= 12) ? load_input3 : 0;
            am_pm <= 0; // Default to AM
        end
    end
endmodule

```

```

end else if (sec == 59) begin
    sec <= 0;
    if (min == 59) begin
        min <= 0;
        if (hours == 11) begin
            hours <= 12;
            am_pm <= ~am_pm;
        end else if (hours == 12) begin
            hours <= 1;
        end else begin hours <= hours + 1;
        end
    end else begin
        min <= min + 1;
    end
end else begin
    sec <= sec + 1;
end
end
endmodule

```

```

// Seven-Segment Display Decoder
module seven_segment(seg, in);
    input [3:0] in;
    output reg [6:0] seg;
    always @(*) begin
        case (in)
            4'b0000: seg = 7'b1000000; // 0
            4'b0001: seg = 7'b1111001; // 1
            4'b0010: seg = 7'b0100100; // 2
            4'b0011: seg = 7'b0110000; // 3
            4'b0100: seg = 7'b0011001; // 4
            4'b0101: seg = 7'b0010010; // 5
            4'b0110: seg = 7'b0000010; // 6
            4'b0111: seg = 7'b1111000; // 7
            4'b1000: seg = 7'b0000000; // 8
            4'b1001: seg = 7'b0010000; // 9
            default: seg = 7'b1111111; // Off
        endcase
    end
endmodule
module timer(

```

```

    seg_out,
    load_input1,
    load_input2,
    load_input3,
    CLK100MHZ,
    reset,
    an,
    led_out
);
    input CLK100MHZ, reset;
    input [5:0] load_input1, load_input2;
    input [3:0] load_input3;
    output [6:0] seg_out;
    output reg [11:0] led_out;
    output reg [3:0] an;
    wire clk_out;
    wire [1:0] count;
    wire [5:0] sec_1, min_1;
    wire [3:0] hours_1;
    reg [3:0] tens, units, tens_1, units_1;
    wire display_clk;
    wire stop;
    // Frequency divider to create a 1 Hz clock from 100 MHz
    freq1 fq(.clk_in(CLK100MHZ), .clk_out(clk_out));
    display_clock_divider1 display_clk_div(.clk_in(CLK100MHZ), .clk_out(display_clk));

    // Counter module
    timer_count cn(
        .clk(clk_out),
        .rst(reset),
        .sec(sec_1),
        .min(min_1),
        .hours(hours_1),
        .load_input1(load_input1),
        .load_input2(load_input2),
        .load_input3(load_input3),
        .stop(stop)
    );

    // 4-bit multiplexing counter
    count_41 cnn(.clk(display_clk), .rst(reset), .count(count));

```

```

    // LED output logic for hours
always @(*) begin
    led_out = 12'b0; // Reset all LEDs
    if (hours_1 > 0 && hours_1 <= 12) begin
        for (integer i = 0; i < hours_1; i = i + 1) begin
            led_out[i] = 1;
        end
    end
end

// Update BCD values for display
always @(*) begin
if(stop)
begin
    tens_1 = 4'b1010;
    units_1 = 4'b1011;
    tens = 4'b1100;
    units = 4'b1101;
end
else begin
    tens = sec_1 / 10;
    units = sec_1 % 10;
    tens_1 = min_1 / 10;
    units_1 = min_1 % 10;
end
end

// Multiplexing logic for seven-segment display
reg [3:0] bcd;
always @(posedge display_clk) begin
    if (reset) begin
        an = 4'b1111;
        bcd = 4'b0000;
    end else begin
        case (count)
            2'b00: begin
                an = 4'b0111;
                bcd = tens_1; // Tens of minutes
            end
            2'b01: begin
                an = 4'b1011;

```

```

        bcd = units_1; // Units of minutes
    end
    2'b10: begin
        an = 4'b1101;
        bcd = tens; // Tens of seconds
    end
    2'b11: begin
        an = 4'b1110;
        bcd = units; // Units of seconds
    end
    default: begin
        an = 4'b1111;
        bcd = 4'b0000;
    end
endcase
end
end
end

```

```

// Seven-segment display decoder
seven_segment1 ssg(.seg(seg_out), .in(bcd));
endmodule

```

```

// Frequency Divider
module freq1(clk_in, clk_out);
    input clk_in;
    output reg clk_out;
    reg [26:0] counter;
    initial begin
        clk_out = 0;
        counter = 0;
    end

    always @(posedge clk_in) begin
        if (counter == 50000000) begin
            counter <= 0;
            clk_out <= ~clk_out;
        end else begin
            counter <= counter + 1;
        end
    end
end
endmodule

```



```
// Display Clock Divider
module display_clock_divider1(input wire clk_in, output reg clk_out);
    reg [16:0] counter;
    always @(posedge clk_in) begin
        if (counter == 10000) begin
            clk_out <= ~clk_out;
            counter <= 0;
        end else begin
            counter <= counter + 1;
        end
    end
end
endmodule
```

```
// 4-bit counter for multiplexing
module count_41(input clk, rst, output reg [1:0] count);
    always @(posedge clk) begin
        if (rst) begin
            count <= 2'b00;
        end else if (count == 2'b11) begin
            count <= 2'b00;
        end else begin
            count <= count + 1;
        end
    end
end
endmodule
```

```
// timer Counter Module
module timer_count(clk,rst,load_input1,load_input2,load_input3,sec,min,hours,stop);
    input clk,rst;
    input [5:0] load_input1,load_input2;
    input [3:0] load_input3;
    output reg [5:0] sec,min;
    output reg [3:0] hours;
    output reg stop;
    always @(posedge clk) begin
        if (rst) begin
            sec <= (load_input1 < 60) ? load_input1 : 0;
            min <= (load_input2 < 60) ? load_input2 : 0;
            hours <= (load_input3 >= 0 && load_input3 <= 12) ? load_input3 : 0 ;
            stop<=0;
        end
    end
endmodule
```

```

end else if (sec == 0) begin
    sec = 59;
    if (min == 0) begin
        min = 59;
        if (hours == 0 ) begin
            stop=1;
        end
        else begin
            hours=hours-1;
        end
    end else begin
min <= min - 1;
    end
    end else begin
        sec <= sec - 1;
    end
end
endmodule

```

```

// Seven-Segment Display Decoder
module seven_segment1(seg, in);
    input [3:0] in;
    output reg [6:0] seg;
    always @(*) begin
        case (in)
            4'b0000: seg = 7'b1000000; // 0
            4'b0001: seg = 7'b1111001; // 1
            4'b0010: seg = 7'b0100100; // 2
            4'b0011: seg = 7'b0110000; // 3
            4'b0100: seg = 7'b0011001; // 4
            4'b0101: seg = 7'b0010010; // 5
            4'b0110: seg = 7'b0000010; // 6
            4'b0111: seg = 7'b1111000; // 7
            4'b1000: seg = 7'b0000000; // 8
            4'b1001: seg = 7'b0010000; // 9
            4'b1010: seg = 7'b0010010; // s
            4'b1011: seg = 7'b0000111; // t
            4'b1100: seg = 7'b1000000; // o
            4'b1101: seg = 7'b0001100; // p
            default: seg = 7'b1111111; // Off
        endcase
    end
endmodule

```

```

        endcase
    end
endmodule

module stop_watch(start,stop,clk,seg_out,an,led_out);
    input start,clk,stop;
    output [6:0] seg_out;
    output reg [3:0] an;
    output reg [11:0] led_out;

    wire clk_out;
    wire [1:0] count;
    wire [5:0] sec_1, min_1;
    wire [3:0] hours_1;
    reg [3:0] tens, units, tens_1, units_1;
    wire display_clk;

    // Frequency divider to create a 1 Hz clock from 100 MHz
    freq2 fq(.clk_in(clk), .clk_out(clk_out));
    display_clock_divider2 display_clk_div(.clk_in(clk), .clk_out(display_clk));

    // Counter module
    stopwatch_count cn(
        .clk(clk_out),
        .rst(start),
        .sec(sec_1),
        .min(min_1),
        .hours(hours_1),
        .stop(stop)
    );
    // 4-bit multiplexing counter
    count_42 cnn(.clk(display_clk), .rst(start), .count(count));

    // Update BCD values for display
    always @(*) begin
        tens = sec_1 / 10;
        units = sec_1 % 10;
        tens_1 = min_1 / 10;
        units_1 = min_1 % 10;
    end

    // LED output logic for hours

```

```

always @(*) begin
    led_out = 12'b0; // Reset all LEDs
    if (hours_1 > 0 && hours_1 <= 12) begin
        for (integer i = 0; i < hours_1; i = i + 1) begin
            led_out[i] = 1;
        end
    end
end

// Multiplexing logic for seven-segment display
reg [3:0] bcd;
always @(posedge display_clk) begin
    if (start==0) begin
        an = 4'b0000;
        bcd = 4'b0000;
    end else begin
        case (count)
            2'b00: begin
an = 4'b0111;
                bcd = tens_1; // Tens of minutes
            end
            2'b01: begin
                an = 4'b1011;
                bcd = units_1; // Units of minutes
            end
            2'b10: begin
                an = 4'b1101;
                bcd = tens; // Tens of seconds
            end
            2'b11: begin
                an = 4'b1110;
                bcd = units; // Units of seconds
            end
            default: begin
                an = 4'b1111;
                bcd = 4'b0000;
            end
        endcase
    end
end
end

```

```

// Seven-segment display decoder
seven_segment2 ssg(.seg(seg_out), .in(bcd))
endmodule

```

```

// Frequency Divider
module freq2(clk_in, clk_out);
    input clk_in;
    output reg clk_out;
    reg [26:0] counter;

    initial begin
        clk_out = 0;
        counter = 0;
    end

    always @(posedge clk_in) begin
        if (counter == 50000000) begin
            counter <= 0;
            clk_out <= ~clk_out;
        end else begin
            counter <= counter + 1;
        end
    end
endmodule

```

```

// Display Clock Divider
module display_clock_divider2(input wire clk_in, output reg clk_out);
    reg [16:0] counter;
    always @(posedge clk_in) begin
        if (counter == 10000) begin
            clk_out <= ~clk_out;
            counter <= 0;
        end else begin
            counter <= counter + 1;
        end
    end
endmodule

```

```

// 4-bit counter for multiplexing
module count_42(input clk, rst, output reg [1:0] count);
    always @(posedge clk) begin
        if (rst==0) begin

```

```

        count <= 2'b00;
    end else if (count == 2'b11) begin
        count <= 2'b00;
    end else begin
        count <= count + 1;
    end
end
endmodule

```

```

// Counter Module
module stopwatch_count(
    input clk,rst,stop,
    output reg [5:0] sec,min,
    output reg [3:0] hours
);
    always @(posedge clk) begin
        if(stop)begin
            sec<=sec;
            min<=min;
            hours<=hours;
        end
        else begin
            if (rst==0) begin
                sec = 0;
                min = 0;
                hours =0;
            end else if (sec == 59) begin
                sec <= 0;
                if (min == 59) begin
                    min <= 0;
                    if (hours == 11) begin
                        hours <= 12;
                    end else if (hours == 12) begin
                        hours <= 1;
                    end else begin
                        hours <= hours + 1;
                    end
                end else begin
                    min <= min + 1;
                end
            end else begin

```

```

        sec <= sec + 1;
    end
end
end
endmodule

// Seven-Segment Display Decoder
module seven_segment2(seg, in);
    input [3:0] in;
    output reg [6:0] seg;
    always @(*) begin
case (in)
    4'b0000: seg = 7'b1000000; // 0
    4'b0001: seg = 7'b1111001; // 1
    4'b0010: seg = 7'b0100100; // 2
    4'b0011: seg = 7'b0110000; // 3
    4'b0100: seg = 7'b0011001; // 4
    4'b0101: seg = 7'b0010010; // 5
    4'b0110: seg = 7'b0000010; // 6
    4'b0111: seg = 7'b1111000; // 7
    4'b1000: seg = 7'b0000000; // 8
    4'b1001: seg = 7'b0010000; // 9
    default: seg = 7'b1111111; // Off
endcase
    end
endmodule

module clock_timer_stopwatch(
    f_seg_out,
    f_load_input1,
    f_load_input2,
    f_load_input3,
    f_CLK100MHZ,
    f_reset,
    f_an,
    f_led_out,
    f_am_pm,
    f_clock,
    f_timer,
    f_stopwatch
);
    input f_CLK100MHZ, f_reset, f_clock, f_timer, f_stopwatch;

```

```

input [5:0] f_load_input1, f_load_input2;
input [3:0] f_load_input3;
output [6:0] f_seg_out;
output [3:0] f_an;
output [11:0] f_led_out;
output f_am_pm;

reg [1:0] mode; // 2-bit mode: 00=Clock, 01=Timer, 10=Stopwatch

// Control flags for functionality
always @(posedge f_CLK100MHZ or posedge f_reset) begin
    if (f_reset) begin
mode <= 2'b00; // Default to clock
        end else begin
            case ({f_stopwatch, f_timer, f_clock})
                3'b001: mode <= 2'b00; // Clock mode
                3'b010: mode <= 2'b01; // Timer mode
                3'b100: mode <= 2'b10; // Stopwatch mode
                default: mode <= mode; // Retain last mode if multiple are selected
            endcase
        end
    end
end

// Instantiate modules based on mode
wire [6:0] clock_seg, timer_seg, stopwatch_seg;
wire [3:0] clock_an, timer_an, stopwatch_an;
wire [11:0] clock_led, timer_led, stopwatch_led;
wire clock_am_pm;

// Clock Module
digital_clock_12hours clock_inst(
    .seg_out(clock_seg),
    .am_pm(clock_am_pm),
    .led_out(clock_led),
    .load_input1(f_load_input1),
    .load_input2(f_load_input2),
    .load_input3(f_load_input3),
    .CLK100MHZ(f_CLK100MHZ),
    .reset(f_reset),
    .an(clock_an)
);

```



```

// Timer Module
timer timer_inst(
    .seg_out(timer_seg),
    .load_input1(f_load_input1),
    .load_input2(f_load_input2),
    .load_input3(f_load_input3),
    .CLK100MHZ(f_CLK100MHZ),
    .reset(f_reset),
    .an(timer_an),
    .led_out(timer_led)
);

// Stopwatch Module
stop_watch stopwatch_inst(
    .start(f_load_input1[0]),
    .stop(f_load_input1[1]),
    .clk(f_CLK100MHZ),
    .seg_out(stopwatch_seg),
    .an(stopwatch_an),
    .led_out(stopwatch_led)
);

// Output logic
assign f_seg_out = (mode == 2'b00) ? clock_seg :
    (mode == 2'b01) ? timer_seg :
    stopwatch_seg;
assign f_an = (mode == 2'b00) ? clock_an :
    (mode == 2'b01) ? timer_an :
    stopwatch_an;

assign f_led_out = (mode == 2'b00) ? clock_led :
    (mode == 2'b01) ? timer_led :
    stopwatch_led;

assign f_am_pm = (mode == 2'b00) ? clock_am_pm : 1'b0;

endmodule

```

Conclusion

In this mini project, we successfully designed and implemented a digital clock, a timer, and a stopwatch using an FPGA (Field Programmable Gate Array) board. These three systems are widely used in our everyday lives, and building them using an FPGA helped us understand how real-time digital systems work at the hardware level.

Throughout the project, we explored how time-based devices operate and how they can be designed using digital logic. Instead of using software alone, we used the power of hardware design through FPGA programming. This gave us hands-on experience with creating circuits that are fast, accurate, and reliable.

The digital clock shows the current time in hours, minutes, and seconds, just like the ones we use at home or in public places. The timer counts down from a set value and is useful in applications like cooking, industrial machines, or fitness routines. The stopwatch counts up from zero and is perfect for sports events, experiments, and any situation where precise time measurement is needed.

Using the FPGA board allowed us to build these systems in a flexible and reprogrammable way. We used counters, registers, and clock dividers to keep track of time accurately. With additional logic, we managed start, stop, and reset functions, as well as displayed the output on 7-segment displays or other output devices.

One of the biggest advantages of this project was learning how to write hardware description language (HDL) code such as Verilog or VHDL. This taught us how to think in terms of digital circuits instead of traditional software programming. We also improved our understanding of concepts like clock signals, timing control, and state machines.

In conclusion, this mini project gave us a strong foundation in both digital system design and practical time-based applications. It showed us how simple concepts like clocks and timers are built and used in real-world electronics. More importantly, it helped us build useful skills that are valuable in fields like embedded systems, digital electronics, and FPGA development. This project was a great combination of learning, designing, and implementing real-life systems using modern technology.